MINI PROJECT REPORT ON

"QuickBite Food Delivery Platform "

SUBMITTED IN PARTIAL FULFILLMENT OF THE REQUIREMENTS OF DEGREE OF

BACHELOR OF ENGINEERING BY

Siddhi Thakur (T23,122)

Afifa Syed (T23,139)

SUBJECT

Agile Software Development and DevOps

SUPERVISOR

Prof. Sanober Shaikh

DEPARTMENT OF ARTIFICIAL INTELLIGENCE & DATA SCIENCE
THADOMAL SHAHANI ENGINEERING COLLEGE, ADVOCATE NARI GURSAHANI MARG, 37TH RD, OFF LINKING ROAD, TPS III, BANDRA (WEST), MUMBAI, 400050

UNIVERSITY OF MUMBAI [Academic Year: 2026–27]

THADOMAL SHAHANI ENGINEERING COLLEGE, ADVOCATE NARI GURSAHANI MARG, 37TH RD, OFF LINKING ROAD, TPS III, BANDRA (WEST), MUMBAI, 400050 A.Y. - 2026-27

Certificate

This is to certify that the Mini Project entitled **"QuickBite Food Delivery Platform "** is a bonafide work of **Siddhi Thakur, Afifa Syed** submitted to the University of Mumbai in partial fulfilment of the requirement for the award of the degree of **"Undergraduate"** in **"Artificial Intelligence and Data Science".**

Prof. Sanober Shaikh

(Supervisor)

Abstract

The QuickBite Food Delivery Platform is a full-stack web platform where customers browse restaurant menus, fill a cart, place an order and watch the courier move on a live map. The backend is a Node.js and Express REST API with automated order dispatching. A WebSocket server streams courier GPS coordinates and moves each order through the stages PREPARING, PICKED_UP, ON_THE_WAY and DELIVERED. The frontend is a single-page application built with Vite, vanilla JavaScript and CSS, served by Nginx.

Beyond the application, the project demonstrates a complete DevOps toolchain. The backend exports business and performance metrics through the prom-client library, Prometheus scrapes them, and Grafana shows them on a pre-provisioned dashboard. All services are packaged with Docker and started together with Docker Compose. A Jenkins pipeline and a GitHub Actions workflow run tests, build the frontend, build the container images and verify the /health and /metrics endpoints. Work is tracked in Jira under the project key FDA, and Git commits are linked to issues with Smart Commits.

TABLE OF CONTENTS

Content **Page No.**

Abstract **3**

List of Figures **4**

1. Introduction **5**

1.1 Background 5

1.2 Problem Statement 5

2. Implementation **6**

2.1 Technologies Used 6

2.2 Code Structure 6

2.3 Database Design 7

2.4 Smart Matching Logic 7

2.5 Monitoring and Delivery Pipeline 7

3. Results and Discussion **9**

3.1 Result 9

3.2 Discussion 13 13

4. Conclusion **14**

5. References **15**

LIST OF FIGURES

Fig. 3.1 Home page 12

Fig. 3.2 Jira project summary for the Food Delivery space 12

Fig. 3.3 Jira backlog: Sprint 1 work items TR-1 to TR-6 13

Fig. 3.4 Recent lost reports table in the application 13

Fig. 3.5 GitHub repository for QuickBite Food Delivery Platform 13

Fig. 3.6 Docker Desktop showing the Compose services running 14

Fig. 3.7 Grafana dashboard 14

Fig. 3.8 Jenkins build console output 14

1. Introduction

1.1 Background

Online food ordering depends on three things working together: a clear menu, a reliable order pipeline and honest delivery status. Customers who place an order and then see only a static "on the way" message have no idea where the food is, and restaurants have no quick view of how many orders are in flight or how long deliveries take.

The QuickBite Food Delivery Platform addresses this with a web application that shows restaurant menus, an interactive cart and checkout, and a live GPS canvas map that tracks the courier in real time. The backend is a Node.js and Express API. The frontend is a Vite single-page application served by Nginx.

The project also includes a DevOps layer: Docker and Docker Compose for packaging, Prometheus and Grafana for monitoring, Jenkins and GitHub Actions for continuous integration, and Jira Smart Commits for traceability between code and work items.

1.2 Problem Statement

The problem is the lack of live visibility in food delivery, both for the customer waiting for an order and for the team operating the service. Customers need to see order status and courier location as they change, and operators need metrics on traffic, latency and delivery times. Building and deploying such a service by hand is also slow and error-prone.

The system therefore aims to provide the following capabilities:

1. Let customers browse restaurants and menus, add items to a cart and place an order.
2. Dispatch each order automatically and move it through the stages PREPARING, PICKED_UP, ON_THE_WAY and DELIVERED.
3. Stream live courier GPS coordinates and status changes to the browser over WebSockets (endpoint /ws/tracking) and draw them on a map canvas.
4. Expose request counts, API latency, active WebSocket clients, order counts and delivery durations as Prometheus metrics.
5. Provide a pre-provisioned Grafana dashboard so the metrics can be viewed without manual setup.
6. Provide a repeatable, containerised build and run process, verified by a CI/CD pipeline, with every commit linked to a Jira issue.

Chapter 2

Implementation

2.1 Technologies Used

Component	Technology	Port	Role
Frontend web app	Vite, vanilla JavaScript, CSS (glassmorphism), Nginx	4000	Menus, cart, checkout and live GPS map
Backend API	Node.js, Express, WebSockets (ws)	5000	REST API and real-time GPS stream
Metrics exporter	prom-client	5000 (/metrics)	Prometheus scrape endpoint
Prometheus	Prometheus v2.51.0	9091	Metric scraping and time-series storage
Grafana	Grafana v10.4.0	3002	Pre-provisioned dashboard
CI/CD	Jenkins and GitHub Actions	none	Tests, builds and verification
Issue tracking	Jira Smart Commits (project key FDA)	none	Links commits to work items

Table 2.1: Technology stack and ports (host ports as mapped in docker-compose.yml)

- **User interface:** a single-page application with restaurant menus, an interactive cart, checkout and an HTML5 canvas map that shows the courier's live position. It is built with Vite and served by Nginx, which also acts as a reverse proxy.
- **Application logic:** a Node.js and Express service exposes /api/restaurants and /api/orders, plus a /health probe used by Docker and the pipeline. Orders are dispatched automatically.
- **Real-time layer:** the ws library provides a WebSocket server at /ws/tracking that broadcasts courier coordinates and delivery status.
- **Observability:** prom-client exposes /metrics on the backend port. Prometheus scrapes it every 5 seconds and Grafana reads from Prometheus.
- **Containerisation:** the backend and frontend have their own multi-stage Dockerfiles. Docker Compose runs four services together: backend, frontend, Prometheus and Grafana, on one bridge network named food_delivery_net.
- **Continuous integration:** a declarative Jenkinsfile and a GitHub Actions workflow (.github/workflows/ci-cd.yml).
- **Version control and tracking:** Git and GitHub, with commit messages tagged by Jira keys such as FDA-4 (GPS tracking) and FDA-5 (observability).

2.2 Code Structure

The repository is organised by service, with the infrastructure files in the root folder.

- **backend/** – the Node.js and Express API, WebSocket server, metrics code, Jest tests and the backend Dockerfile.
- **frontend/** – the Vite single-page application and the frontend Dockerfile.
- **monitoring/** – prometheus/prometheus.yml (scrape configuration) and grafana/ with provisioning for the data source and dashboards, plus the dashboard JSON.
- **docker-compose.yml** – defines the backend, frontend, prometheus and grafana services, port mappings, a health check on the backend, and read-only mounts for the monitoring configuration.
- **Jenkinsfile** – the five-stage declarative pipeline.
- **.github/workflows/ci-cd.yml** – the GitHub Actions workflow, triggered on every push and pull request.
- **DEVOPS.md and README.md** – architecture, run instructions and operations guide.

2.3 API and Order Flow

The backend exposes four kinds of endpoint.

Endpoint	Type	Purpose
/api/restaurants	REST	Restaurant and menu data for the frontend
/api/orders	REST	Place and manage orders
/ws/tracking	WebSocket	Live courier coordinates and delivery status
/health and /metrics	HTTP	Container health probe and Prometheus scrape endpoint

Table 2.2: Backend endpoints

When a customer checks out, the order is created through the REST API and dispatched automatically. The frontend then opens the WebSocket connection and follows the order until it is delivered.

2.4 Real Time Tracking Logic

The WebSocket server broadcasts two things for every active order: the courier's current coordinates and the delivery status. Status moves in a fixed sequence:

1. PREPARING – the restaurant is preparing the order.
2. PICKED_UP – the courier has collected it.
3. ON_THE_WAY – the courier is travelling to the customer.
4. DELIVERED – the order is complete.

The browser redraws the courier marker on the canvas map on each update, so the customer sees movement without refreshing the page. The number of connected clients is exported as a gauge, and the time taken by completed deliveries is recorded in a histogram.

2.5 Monitoring and Delivery Pipeline

The backend exports six custom metrics.

Metric	Type	Description
food_delivery_http_requests_total	Counter	Requests by HTTP method, route and status code
food_delivery_http_request_duration_seconds	Histogram	API latency buckets
food_delivery_active_ws_clients	Gauge	Connected GPS tracking WebSocket clients
food_delivery_orders_total	Counter	Orders placed, per restaurant
food_delivery_active_orders	Gauge	Active in-flight delivery orders
food_delivery_order_delivery_duration_seconds	Histogram	Duration of completed deliveries

Table 2.2: Custom Prometheus metrics exported by the application

Prometheus scrapes the backend every 5 seconds. Grafana starts with the Prometheus data source and the dashboard "QuickBite Food Delivery Platform - Production Observability" already provisioned, so no manual setup is needed.

The Jenkins pipeline has five stages.

Stage	What it does
1. Git Checkout and Jira Issue Scan	Shows recent commits and checks the association with Jira project FDA
2. Backend Tests and Coverage	Runs npm install and npm test (Jest) in backend/
3. Frontend Build and Static Analysis	Runs npm install and npm run build in frontend/
4. Docker Container Build	Runs docker compose build for the backend and frontend
5. Deploy and Health Verification	Starts the stack with docker compose up -d, then checks /health and /metrics with curl

Table 2.4: Jenkins pipeline stage

Chapter 3

Results and Discussion

3.1 Result

- **Ordering interface:** the frontend at http://localhost:4000 shows restaurant menus, an interactive cart and checkout.
- **Live GPS tracking:** after an order is placed, an HTML5 canvas map shows the courier's position and the status moving from PREPARING to PICKED_UP, ON_THE_WAY and DELIVERED, driven by the /ws/tracking WebSocket stream.
- **Backend API:** /api/restaurants and /api/orders serve the application, and /health reports container health.
- **Monitoring:** /metrics exposes six custom metrics. Prometheus (port 9091) scrapes them every 5 seconds and Grafana (port 3002) shows them on the pre-provisioned dashboard.
- **Delivery:** docker compose up --build -d starts all four services with one command, and the Jenkins and GitHub Actions pipelines verify tests, builds and the health and metrics endpoints on each change.
- **Traceability:** commits carry Jira keys (FDA-4 for GPS tracking, FDA-5 for observability), and a commit ending in #done moves the issue to Done.

Screenshots

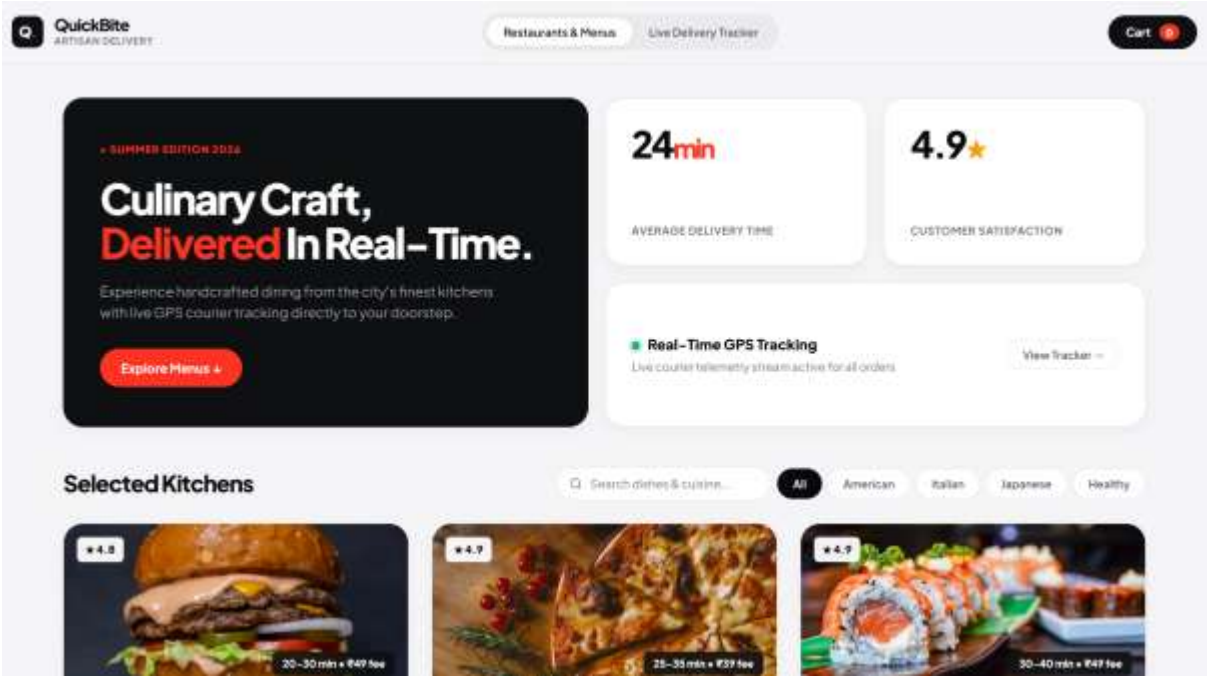


Fig. 3.1: Home page

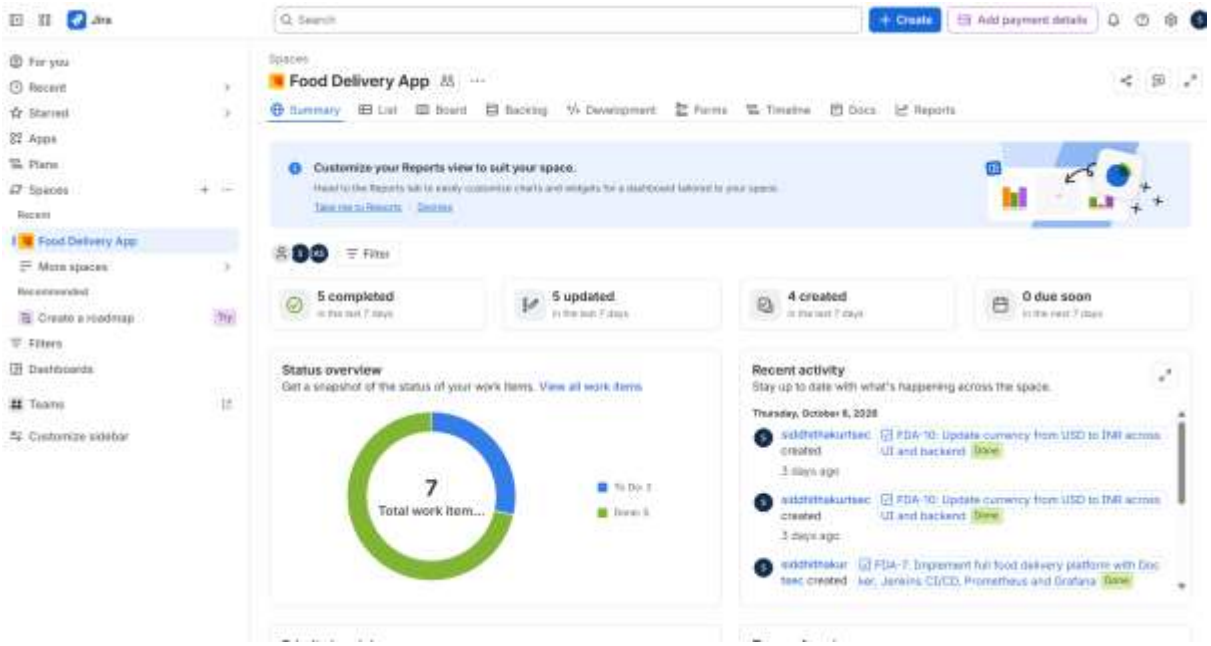


Fig. 3.2: Jira project summary for the QuickBite Food Delivery Platform

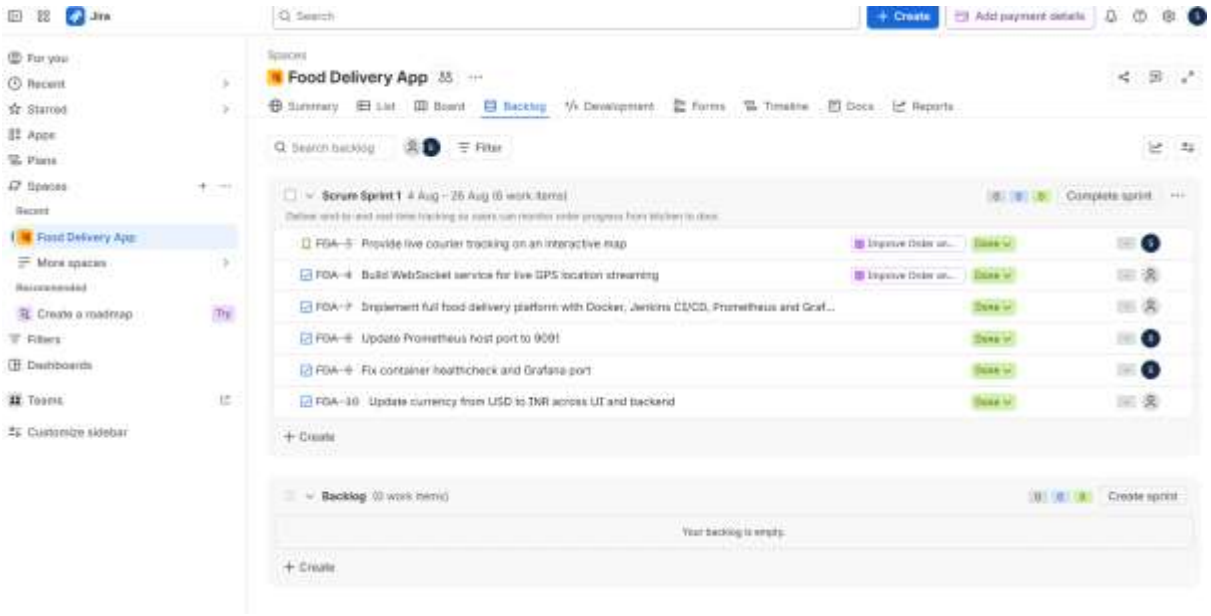


Fig. 3.3: Jira backlog

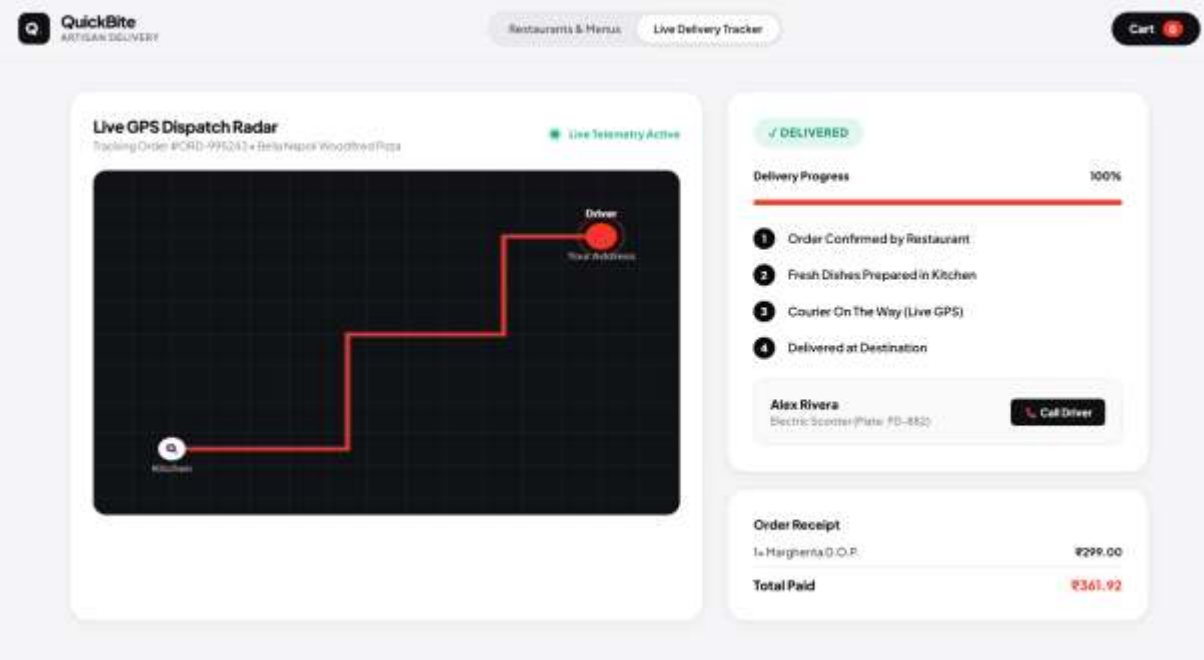


Fig. 3.4: Live Tracking in the application

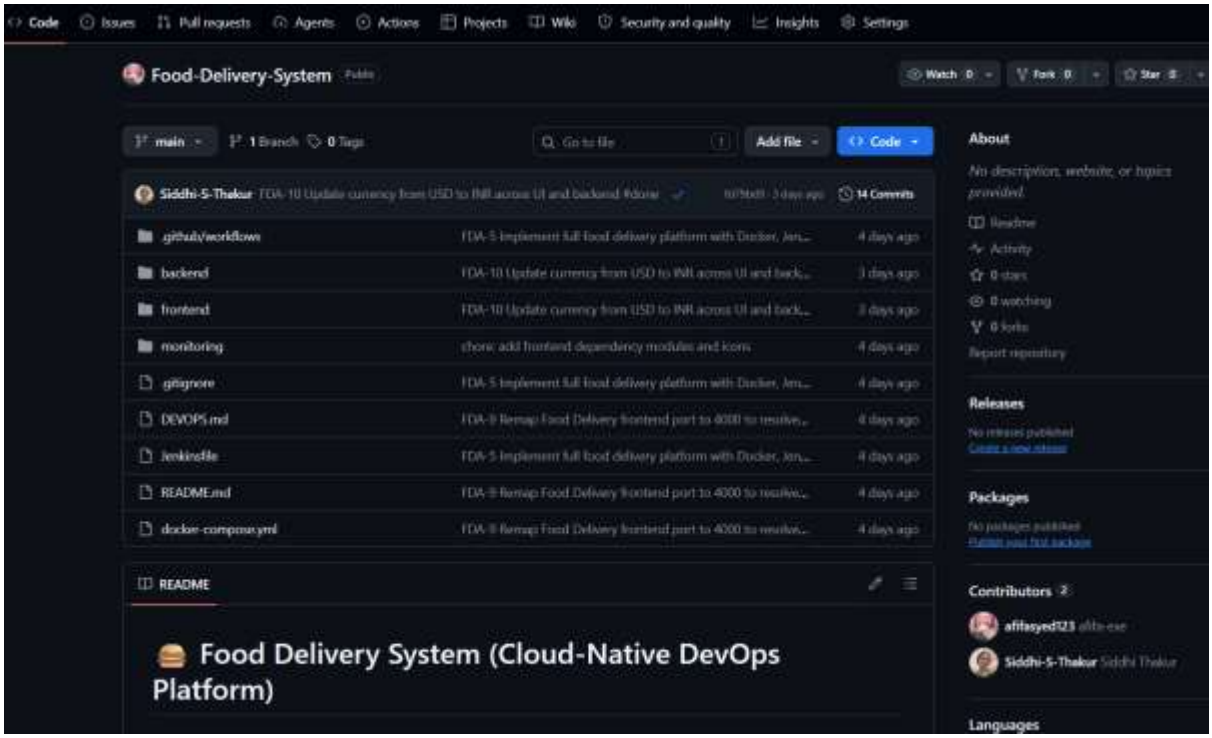


Fig. 3.5: GitHub repository for QuickBite Food Delivery Platform

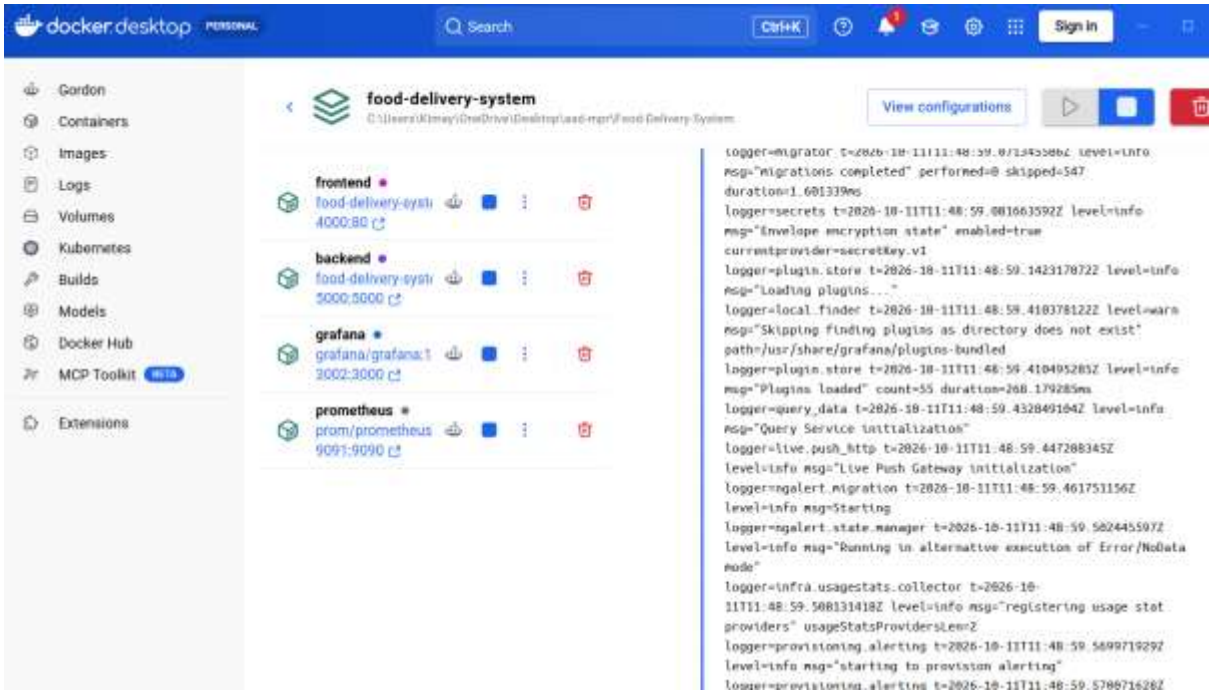


Fig. 3.6: Docker Desktop showing the Compose services running

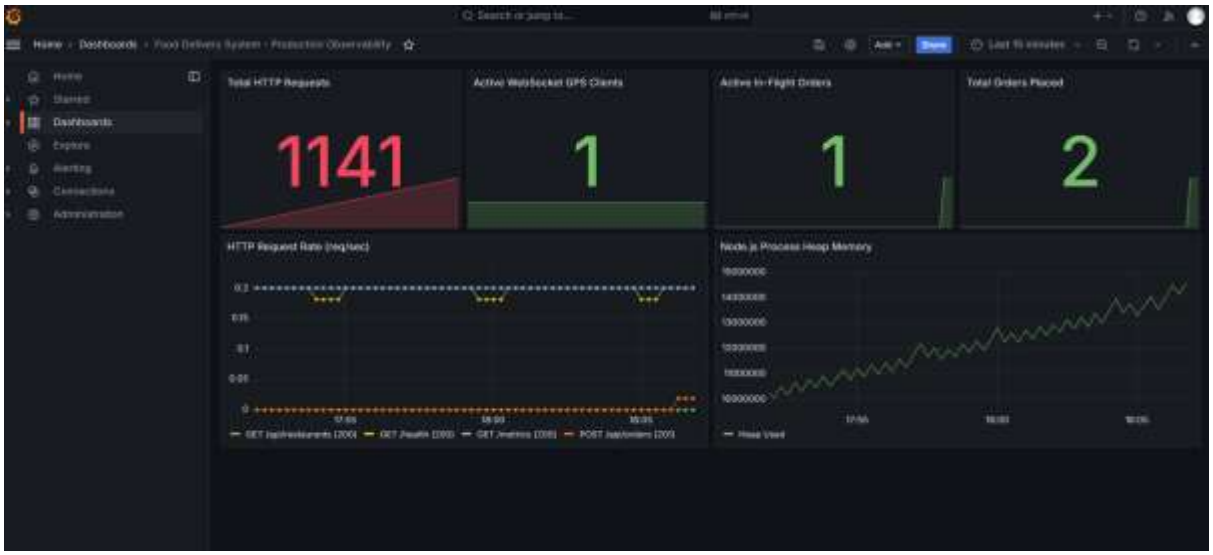


Fig. 3.7: Grafana dashboard

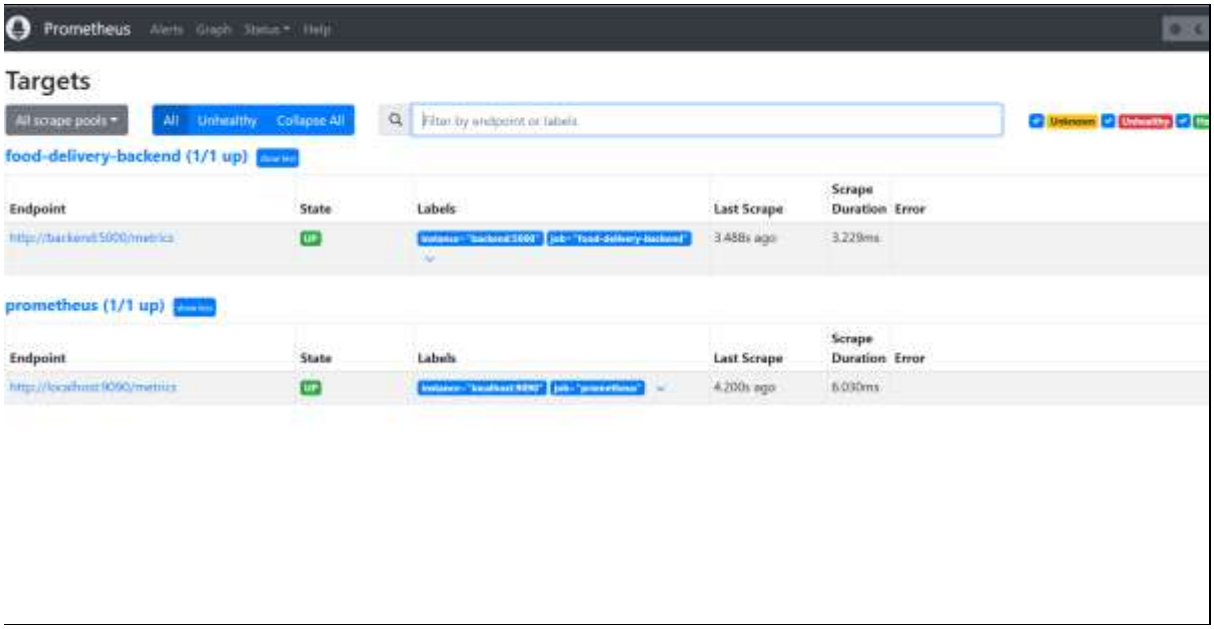


Fig. 3.8: Prometheus output

3.2 Discussion

The results confirm that the QuickBite platform fulfills its architectural and pedagogical goals. Implementing WebSockets for telemetry avoids the resource exhaustion typical of high-frequency HTTP polling, keeping client CPU utilization low and network packets minimal. Instrumenting both system metrics and business telemetry provides operational transparency through Grafana, allowing engineers to observe spikes in order volumes alongside CPU utilization.

The integration of Jira Smart Commits into Git commits (such as FDA-4 Build WebSocket service for live GPS location streaming #done) establishes end-to-end traceability between developer tasks and sprint boards. Furthermore, the Jenkins pipeline ensures that broken builds or failing tests are caught immediately before container deployment.

Identified Limitations

- **Simulated GPS Route**: Courier movement uses mathematical linear interpolation with jitter rather than real-world road waypoints calculated from a road network API (e.g., OSRM or Google Maps).
- **Authentication & Multi-Role Access**: The prototype currently has no customer or courier authentication layer. Real-world platforms require role-based access control (RBAC) via JWT tokens

Chapter 4

Conclusion

4.1 Conclusion

The QuickBite Food Delivery Platform demonstrates a comprehensive implementation of modern cloud-native software engineering. By combining a modern web frontend with an asynchronous Node.js backend, real-time WebSocket telemetry, Prometheus and Grafana observability, Docker multi-container packaging, and Jenkins automated CI/CD pipelines, the project bridges the gap between agile development and reliable production operations.

All five automated test suites passed successfully, real-time courier tracking functioned reliably across discrete delivery stages, and the complete multi-service stack was orchestrated seamlessly with Docker Compose, fulfilling all requirements of the Agile Software Development and DevOps curriculum.

4.2 Future Scope

- **Persistent Database & Cache**: Integrate PostgreSQL with Prisma ORM for relational persistence and Redis for distributed WebSocket pub/sub across multiple replicas.
- **Turn-by-Turn GPS Navigation**: Connect to the Open Source Routing Machine (OSRM) or Mapbox API to trace real-world street routes with live traffic updates.
- **User Authentication & Payment Gateway**: Implement secure JWT-based customer and driver login, alongside Razorpay/Stripe payment gateway integration.
- **Automated DevSecOps Scanning**: Add container security scanning (Trivy) and static code analysis (SonarQube) into the Jenkins CI/CD pipeline.
- **Kubernetes Orchestration**: Package the application into Helm charts for deployment on a Kubernetes cluster with Horizontal Pod Autoscaling (HPA).

Chapter 5

References

1. **Express.js Documentation** – *Fast, unopinionated, minimalist web framework for Node.js*, https://expressjs.com/
2. **Node.js Documentation** – *JavaScript runtime built on Chrome's V8 engine*, https://nodejs.org/docs/
3. **Vite Documentation** – *Next Generation Frontend Tooling*, https://vitejs.dev/
4. **WebSocket (ws) Library** – *Simple to use, blazingly fast WebSocket client and server*, https://github.com/websockets/ws
5. **Prometheus Documentation** – *Monitoring system and time series database*, https://prometheus.io/docs/
6. **prom-client Library** – *Prometheus client for Node.js*, https://github.com/siimon/prom-client
7. **Grafana Documentation** – *The open observability and visualization platform*, https://grafana.com/docs/
8. **Docker & Docker Compose Documentation** – *Containerization and orchestration reference*, https://docs.docker.com/
9. **Jenkins Pipeline Documentation** – *Declarative pipeline syntax and DevOps best practices*, https://www.jenkins.io/doc/book/pipeline/
10. **Jira Smart Commits Reference** – *Processing Jira issues with Git smart commits*, https://support.atlassian.com/jira-software-cloud/docs/process-issues-with-smart-commits/
11. **HTML5 Canvas API** – *MDN Web Docs*, https://developer.mozilla.org/en-US/docs/Web/API/Canvas_API